

Task 2: Stock Trading Platform

Complete Java Swing project source code

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.*;
import java.util.ArrayList;

// =====
// TASK 2: STOCK TRADING PLATFORM
// =====

public class StockTradingPlatform extends JFrame {

    static class User {
        String name;
        double balance;

        User(String name, double balance) {
            this.name = name;
            this.balance = balance;
        }
    }

    static class Stock {
        String symbol;
        String companyName;
        double price;

        Stock(String symbol, String companyName, double price) {
            this.symbol = symbol;
            this.companyName = companyName;
            this.price = price;
        }
    }

    static class Transaction {
        String type;
        String symbol;
        int quantity;
        double price;
        double total;

        Transaction(String type, String symbol, int quantity, double price) {
            this.type = type;
            this.symbol = symbol;
            this.quantity = quantity;
            this.price = price;
            this.total = quantity * price;
        }
    }

    static class Portfolio {
        ArrayList<Stock> stocks;
        ArrayList<Integer> quantities;
        ArrayList<Transaction> transactions;

        Portfolio() {
            stocks = new ArrayList<>();
            quantities = new ArrayList<>();
            transactions = new ArrayList<>();
        }

        int findStock(String symbol) {
            for (int i = 0; i < stocks.size(); i++) {
                if (stocks.get(i).symbol.equalsIgnoreCase(symbol)) {
                    return i;
                }
            }
            return -1;
        }

        boolean buyStock(Stock stock, int quantity, User user) {
            double totalCost = stock.price * quantity;

            if (quantity <= 0) return false;
            if (user.balance < totalCost) return false;

            user.balance -= totalCost;

            int index = findStock(stock.symbol);

            if (index == -1) {
                stocks.add(stock);
                quantities.add(quantity);
            } else {
```

```

        int oldQuantity = quantities.get(index);
        quantities.set(index, oldQuantity + quantity);
    }

    transactions.add(
        new Transaction("BUY", stock.symbol, quantity, stock.price)
    );

    return true;
}

boolean sellStock(Stock stock, int quantity, User user) {
    if (quantity <= 0) return false;

    int index = findStock(stock.symbol);
    if (index == -1) return false;

    int ownedQuantity = quantities.get(index);
    if (quantity > ownedQuantity) return false;

    double totalValue = stock.price * quantity;
    user.balance += totalValue;

    int remaining = ownedQuantity - quantity;

    if (remaining == 0) {
        stocks.remove(index);
        quantities.remove(index);
    } else {
        quantities.set(index, remaining);
    }

    transactions.add(
        new Transaction("SELL", stock.symbol, quantity, stock.price)
    );

    return true;
}

double getPortfolioValue() {
    double value = 0;

    for (int i = 0; i < stocks.size(); i++) {
        value += stocks.get(i).price * quantities.get(i);
    }

    return value;
}

void saveToFile(User user) {
    try {
        PrintWriter writer = new PrintWriter(
            new FileWriter("portfolio.txt")
        );

        writer.println("USER=" + user.name);
        writer.println("BALANCE=" + user.balance);

        for (int i = 0; i < stocks.size(); i++) {
            Stock stock = stocks.get(i);

            writer.println(
                "STOCK=" +
                stock.symbol + "," +
                stock.companyName + "," +
                stock.price + "," +
                quantities.get(i)
            );
        }

        writer.close();

        JOptionPane.showMessageDialog(
            null,
            "Portfolio saved successfully!\nFile: portfolio.txt"
        );

    } catch (IOException e) {
        JOptionPane.showMessageDialog(
            null,
            "Error saving portfolio."
        );
    }
}

User user;
Portfolio portfolio;
ArrayList<Stock> marketStocks;

DefaultTableModel marketModel;
DefaultTableModel portfolioModel;
DefaultTableModel transactionModel;

JTable marketTable;
JTable portfolioTable;

```

```

JTable transactionTable;

JLabel balanceLabel;
JLabel portfolioValueLabel;
JLabel totalValueLabel;

JTextField quantityField;

public StockTradingPlatform() {
    user = new User("Student", 100000);
    portfolio = new Portfolio();

    createMarketData();
    createGUI();
    updateGUI();
}

void createMarketData() {
    marketStocks = new ArrayList<>();

    marketStocks.add(
        new Stock("TCS", "Tata Consultancy Services", 3500)
    );
    marketStocks.add(
        new Stock("INFY", "Infosys", 1800)
    );
    marketStocks.add(
        new Stock("RELIANCE", "Reliance Industries", 2900)
    );
    marketStocks.add(
        new Stock("HDFC", "HDFC Bank", 1650)
    );
    marketStocks.add(
        new Stock("WIPRO", "Wipro", 550)
    );
    marketStocks.add(
        new Stock("ITC", "ITC Limited", 480)
    );
    marketStocks.add(
        new Stock("SBIN", "State Bank of India", 850)
    );
    marketStocks.add(
        new Stock("AAPL", "Apple Inc.", 22000)
    );
}

void createGUI() {
    setTitle("Stock Trading Platform - Task 2");
    setSize(900, 650);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);

    JPanel mainPanel = new JPanel(new BorderLayout());

    JPanel headerPanel = new JPanel(new GridLayout(1, 3));

    balanceLabel = new JLabel();
    portfolioValueLabel = new JLabel();
    totalValueLabel = new JLabel();

    headerPanel.add(balanceLabel);
    headerPanel.add(portfolioValueLabel);
    headerPanel.add(totalValueLabel);

    mainPanel.add(headerPanel, BorderLayout.NORTH);

    JTabbedPane tabs = new JTabbedPane();

    JPanel marketPanel = new JPanel(new BorderLayout());

    String[] marketColumns = {
        "Symbol", "Company", "Price (■)"
    };

    marketModel = new DefaultTableModel(marketColumns, 0);
    marketTable = new JTable(marketModel);

    loadMarketTable();

    marketPanel.add(
        new JScrollPane(marketTable),
        BorderLayout.CENTER
    );

    JPanel tradingPanel = new JPanel();

    quantityField = new JTextField(5);

    JButton buyButton = new JButton("BUY");
    JButton sellButton = new JButton("SELL");
    JButton refreshButton = new JButton("Refresh Market");

    tradingPanel.add(new JLabel("Quantity:"));
    tradingPanel.add(quantityField);
    tradingPanel.add(buyButton);
    tradingPanel.add(sellButton);
}

```

```

tradingPanel.add(refreshButton);

marketPanel.add(tradingPanel, BorderLayout.SOUTH);

buyButton.addActionListener(e -> buyStock());
sellButton.addActionListener(e -> sellStock());

refreshButton.addActionListener(e -> {
    loadMarketTable();
    updateGUI();
});

JPanel portfolioPanel = new JPanel(new BorderLayout());

String[] portfolioColumns = {
    "Symbol", "Company", "Price (■)", "Quantity", "Value (■)"
};

portfolioModel = new DefaultTableModel(portfolioColumns, 0);
portfolioTable = new JTable(portfolioModel);

portfolioPanel.add(
    new JScrollPane(portfolioTable),
    BorderLayout.CENTER
);

JPanel transactionPanel = new JPanel(new BorderLayout());

String[] transactionColumns = {
    "Type", "Stock", "Quantity", "Price", "Total"
};

transactionModel = new DefaultTableModel(transactionColumns, 0);
transactionTable = new JTable(transactionModel);

transactionPanel.add(
    new JScrollPane(transactionTable),
    BorderLayout.CENTER
);

JPanel bottomPanel = new JPanel();

JButton saveButton = new JButton("Save Portfolio");
JButton performanceButton = new JButton("Portfolio Performance");
JButton exitButton = new JButton("Exit");

bottomPanel.add(saveButton);
bottomPanel.add(performanceButton);
bottomPanel.add(exitButton);

saveButton.addActionListener(
    e -> portfolio.saveToFile(user)
);

performanceButton.addActionListener(
    e -> showPerformance()
);

exitButton.addActionListener(
    e -> System.exit(0)
);

tabs.addTab("Market", marketPanel);
tabs.addTab("My Portfolio", portfolioPanel);
tabs.addTab("Transactions", transactionPanel);

mainPanel.add(tabs, BorderLayout.CENTER);
mainPanel.add(bottomPanel, BorderLayout.SOUTH);

add(mainPanel);
}

void loadMarketTable() {
    marketModel.setRowCount(0);

    for (Stock stock : marketStocks) {
        marketModel.addRow(
            new Object[] {
                stock.symbol,
                stock.companyName,
                String.format("%.2f", stock.price)
            }
        );
    }
}

void buyStock() {
    int selectedRow = marketTable.getSelectedRow();

    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(
            this, "Please select a stock first."
        );
        return;
    }
}

```

```

int quantity;

try {
    quantity = Integer.parseInt(quantityField.getText());
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(
        this, "Please enter a valid quantity."
    );
    return;
}

Stock stock = marketStocks.get(selectedRow);

boolean success = portfolio.buyStock(stock, quantity, user);

if (success) {
    JOptionPane.showMessageDialog(
        this,
        "Stock purchased successfully!\n\n" +
        "Stock: " + stock.symbol +
        "\nQuantity: " + quantity +
        "\nPrice: ■" + stock.price
    );
} else {
    JOptionPane.showMessageDialog(
        this,
        "Purchase failed!\nCheck balance and quantity."
    );
}

quantityField.setText("");
updateGUI();
}

void sellStock() {
    int selectedRow = marketTable.getSelectedRow();

    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(
            this, "Please select a stock first."
        );
        return;
    }

    int quantity;

    try {
        quantity = Integer.parseInt(quantityField.getText());
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(
            this, "Please enter a valid quantity."
        );
        return;
    }

    Stock stock = marketStocks.get(selectedRow);

    boolean success = portfolio.sellStock(stock, quantity, user);

    if (success) {
        JOptionPane.showMessageDialog(
            this,
            "Stock sold successfully!\n\n" +
            "Stock: " + stock.symbol +
            "\nQuantity: " + quantity +
            "\nPrice: ■" + stock.price
        );
    } else {
        JOptionPane.showMessageDialog(
            this,
            "Sale failed!\nYou may not own enough shares."
        );
    }

    quantityField.setText("");
    updateGUI();
}

void updateGUI() {
    balanceLabel.setText(
        "Cash: ■" +
        String.format("%.2f", user.balance)
    );

    double portfolioValue = portfolio.getPortfolioValue();

    portfolioValueLabel.setText(
        "Portfolio: ■" +
        String.format("%.2f", portfolioValue)
    );

    totalValueLabel.setText(
        "Total: ■" +
        String.format(
            "%.2f",
            user.balance + portfolioValue
        )
    );
}

```

```

    )
    );
    updatePortfolioTable();
    updateTransactionTable();
}

void updatePortfolioTable() {
    portfolioModel.setRowCount(0);

    for (int i = 0; i < portfolio.stocks.size(); i++) {
        Stock stock = portfolio.stocks.get(i);
        int quantity = portfolio.quantities.get(i);
        double value = stock.price * quantity;

        portfolioModel.addRow(
            new Object[] {
                stock.symbol,
                stock.companyName,
                String.format("%.2f", stock.price),
                quantity,
                String.format("%.2f", value)
            }
        );
    }
}

void updateTransactionTable() {
    transactionModel.setRowCount(0);

    for (Transaction transaction : portfolio.transactions) {
        transactionModel.addRow(
            new Object[] {
                transaction.type,
                transaction.symbol,
                transaction.quantity,
                String.format("%.2f", transaction.price),
                String.format("%.2f", transaction.total)
            }
        );
    }
}

void showPerformance() {
    double portfolioValue = portfolio.getPortfolioValue();
    double totalValue = user.balance + portfolioValue;
    double initialInvestment = 100000;

    double profitLoss = totalValue - initialInvestment;
    double percentage =
        (profitLoss / initialInvestment) * 100;

    String message =
        "PORTFOLIO PERFORMANCE\n\n" +
        "Initial Capital: ■" +
        String.format("%.2f", initialInvestment) +
        "\nCash Balance: ■" +
        String.format("%.2f", user.balance) +
        "\nStock Value: ■" +
        String.format("%.2f", portfolioValue) +
        "\nTotal Portfolio Value: ■" +
        String.format("%.2f", totalValue) +
        "\n\nProfit/Loss: ■" +
        String.format("%.2f", profitLoss) +
        "\nReturn: " +
        String.format("%.2f", percentage) + "%";

    JOptionPane.showMessageDialog(
        this,
        message,
        "Portfolio Performance",
        JOptionPane.INFORMATION_MESSAGE
    );
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        StockTradingPlatform platform =
            new StockTradingPlatform();

        platform.setVisible(true);
    });
}
}

```